TASK 2

In [2]:
```
# a
```

In [4]:
```
import pandas as pd
ratings = pd.read_csv("/Users/qayyumkhan/Downloads/ratings_small.csv")

print(ratings.head())
print(ratings.columns)
print(ratings.shape)
```

```
   userId  movieId  rating   timestamp
0       1       31     2.5  1260759144
1       1     1029     3.0  1260759179
2       1     1061     3.0  1260759182
3       1     1129     2.0  1260759185
4       1     1172     4.0  1260759205
Index(['userId', 'movieId', 'rating', 'timestamp'], dtype='object')
(100004, 4)
```

In [8]:
```
!pip install scikit-surprise
```

```
Collecting scikit-surprise
  Downloading scikit_surprise-1.1.4.tar.gz (154 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.12/site-packages (from s
cikit-surprise) (1.4.2)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.12/site-packages (from s
cikit-surprise) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.12/site-packages (from sc
ikit-surprise) (1.13.1)
Building wheels for collected packages: scikit-surprise
  Building wheel for scikit-surprise (pyproject.toml) ... done
  Created wheel for scikit-surprise: filename=scikit_surprise-1.1.4-cp312-cp312-macosx_11_0_arm64.w
hl size=463832 sha256=211f048ce77bcd02f46371664303aeffe43883da85de6617f081783d1cbf18f6
  Stored in directory: /Users/qayyumkhan/Library/Caches/pip/wheels/75/fa/bc/739bc2cb1fbaab6061854e6
cfbb81a0ae52c92a502a7fa454b
Successfully built scikit-surprise
Installing collected packages: scikit-surprise
Successfully installed scikit-surprise-1.1.4
```

In [10]:
```python
from surprise import Dataset, Reader

# Surprise expects: user item rating (timestamp can be ignored)
reader = Reader(line_format='user item rating timestamp', sep=',', skip_lines=1)

data = Dataset.load_from_file(
    "/Users/qayyumkhan/Downloads/ratings_small.csv",
    reader=reader
)

print("Loaded into Surprise!")
```

```
Loaded into Surprise!
```

In [12]:
```python
# c
```

In [16]:
```python
import numpy as np
from surprise import Dataset, Reader, SVD, KNNBasic
```

```
from surprise.model_selection import cross_validate

reader = Reader(line_format='user item rating timestamp', sep=',', skip_lines=1)
data = Dataset.load_from_file(
    "/Users/qayyumkhan/Downloads/ratings_small.csv",
    reader=reader
)

# 1) PMF via SVD
pmf_algo = SVD(biased=False, random_state=42)
pmf_cv = cross_validate(pmf_algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)

pmf_rmse_mean = np.mean(pmf_cv['test_rmse'])
pmf_mae_mean  = np.mean(pmf_cv['test_mae'])

# 2) User-based CF
sim_user = {'name': 'cosine', 'user_based': True}   # user-user similarity
ubcf_algo = KNNBasic(sim_options=sim_user)
ubcf_cv = cross_validate(ubcf_algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)

ubcf_rmse_mean = np.mean(ubcf_cv['test_rmse'])
ubcf_mae_mean  = np.mean(ubcf_cv['test_mae'])

# 3) Item-based CF
sim_item = {'name': 'cosine', 'user_based': False}  # item-item similarity
ibcf_algo = KNNBasic(sim_options=sim_item)
ibcf_cv = cross_validate(ibcf_algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)

ibcf_rmse_mean = np.mean(ibcf_cv['test_rmse'])
ibcf_mae_mean  = np.mean(ibcf_cv['test_mae'])

print("\n===== 5-Fold Average Results =====")
print(f"PMF (SVD biased=False) -> RMSE mean: {pmf_rmse_mean:.4f}, MAE mean: {pmf_mae_mean:.4f}")
print(f"User-based CF          -> RMSE mean: {ubcf_rmse_mean:.4f}, MAE mean: {ubcf_mae_mean:.4f}")
print(f"Item-based CF          -> RMSE mean: {ibcf_rmse_mean:.4f}, MAE mean: {ibcf_mae_mean:.4f}")
```

```
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

                  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean    Std
RMSE (testset)    0.9934  1.0181  1.0047  1.0202  1.0144  1.0101  0.0099
```

```
MAE (testset)     0.7659  0.7875  0.7742  0.7871  0.7843  0.7798  0.0085
Fit time          0.22    0.24    0.24    0.23    0.22    0.23    0.01
Test time         0.02    0.05    0.02    0.02    0.05    0.03    0.01
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Evaluating RMSE, MAE of algorithm KNNBasic on 5 split(s).

                  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean    Std
RMSE (testset)    0.9891  0.9924  1.0068  0.9908  0.9874  0.9933  0.0069
MAE (testset)     0.7659  0.7656  0.7767  0.7649  0.7654  0.7677  0.0045
Fit time          0.05    0.04    0.04    0.04    0.03    0.04    0.00
Test time         0.29    0.33    0.29    0.32    0.28    0.30    0.02
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Evaluating RMSE, MAE of algorithm KNNBasic on 5 split(s).

                  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean    Std
RMSE (testset)    1.0010  0.9845  0.9926  0.9970  0.9974  0.9945  0.0057
MAE (testset)     0.7799  0.7658  0.7727  0.7759  0.7740  0.7736  0.0046
Fit time          1.08    0.92    0.83    0.83    0.79    0.89    0.10
Test time         1.23    1.27    1.30    1.21    1.21    1.24    0.04

===== 5-Fold Average Results =====
```

```
PMF (SVD biased=False) -> RMSE mean: 1.0101, MAE mean: 0.7798
User-based CF          -> RMSE mean: 0.9933, MAE mean: 0.7677
Item-based CF          -> RMSE mean: 0.9945, MAE mean: 0.7736
```

In [18]:
```
# d
```

Based on the 5-fold mean results we computed, **User-based CF has the lowest RMSE and the lowest MAE**, Item-based CF is a close second, and PMF (SVD biased=False) performs worst on both metrics. Since lower RMSE/MAE means better rating predictions, the **best ML model for this movie-rating data is User-based Collaborative Filtering**, with Item-based CF slightly behind and PMF trailing.

In [22]:
```
# e
```

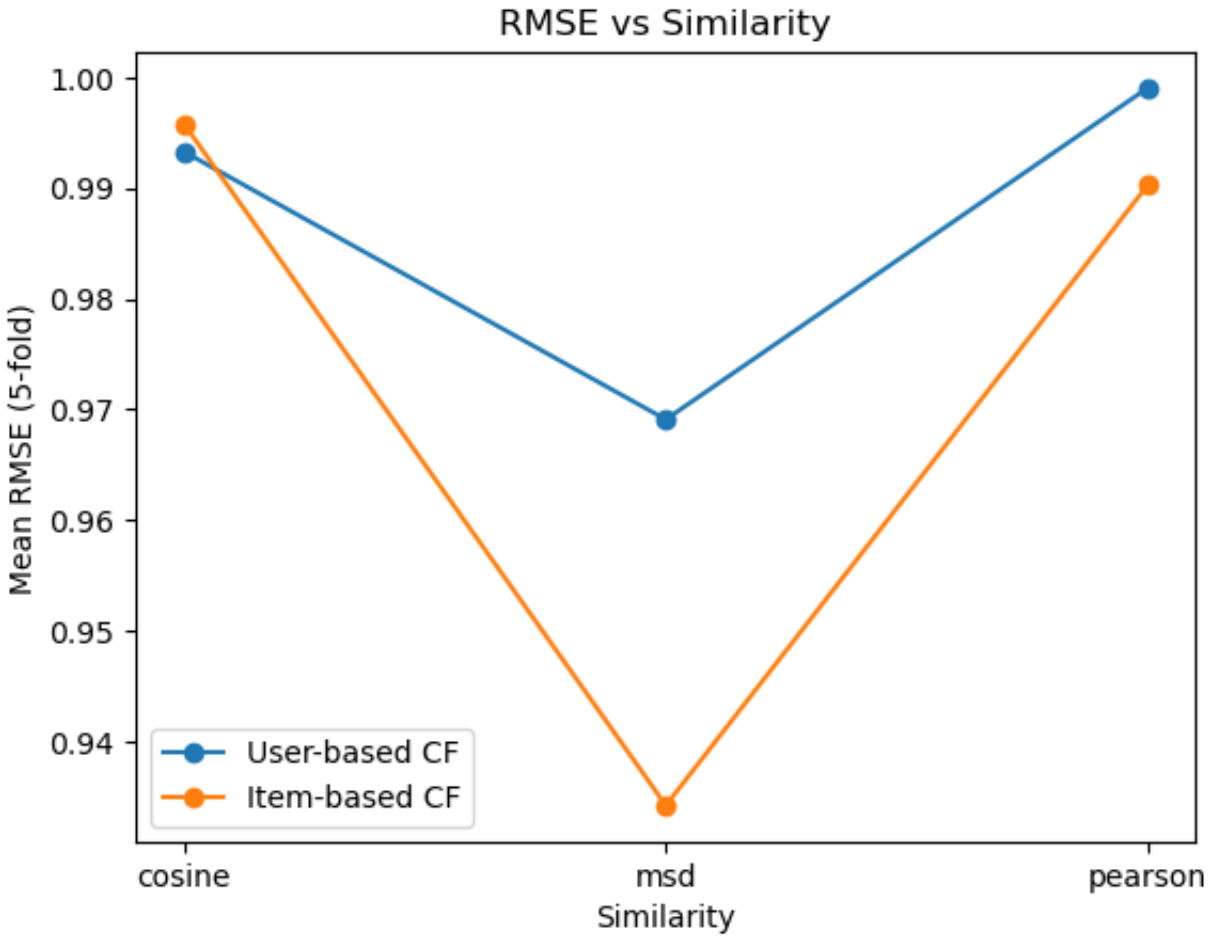
In [24]:
```
import matplotlib.pyplot as plt
from surprise import KNNBasic
from surprise.model_selection import cross_validate

sims = ["cosine", "msd", "pearson"]
K_neighbors = 40

def eval_knn(sim_name, user_based_flag):
    sim_options = {"name": sim_name, "user_based": user_based_flag}
    algo = KNNBasic(k=K_neighbors, sim_options=sim_options)
    cv_res = cross_validate(algo, data, measures=["RMSE", "MAE"], cv=5, verbose=False)
    return np.mean(cv_res["test_rmse"]), np.mean(cv_res["test_mae"])

rows = []
for sim in sims:
    rmse_u, mae_u = eval_knn(sim, True)   # User-based
    rmse_i, mae_i = eval_knn(sim, False)  # Item-based
    rows.append(["User-based CF", sim, rmse_u, mae_u])
    rows.append(["Item-based CF", sim, rmse_i, mae_i])

df_e = pd.DataFrame(rows, columns=["Model", "Similarity", "RMSE_mean", "MAE_mean"])
print(df_e)

# Plottinng RMSE
plt.figure()
```

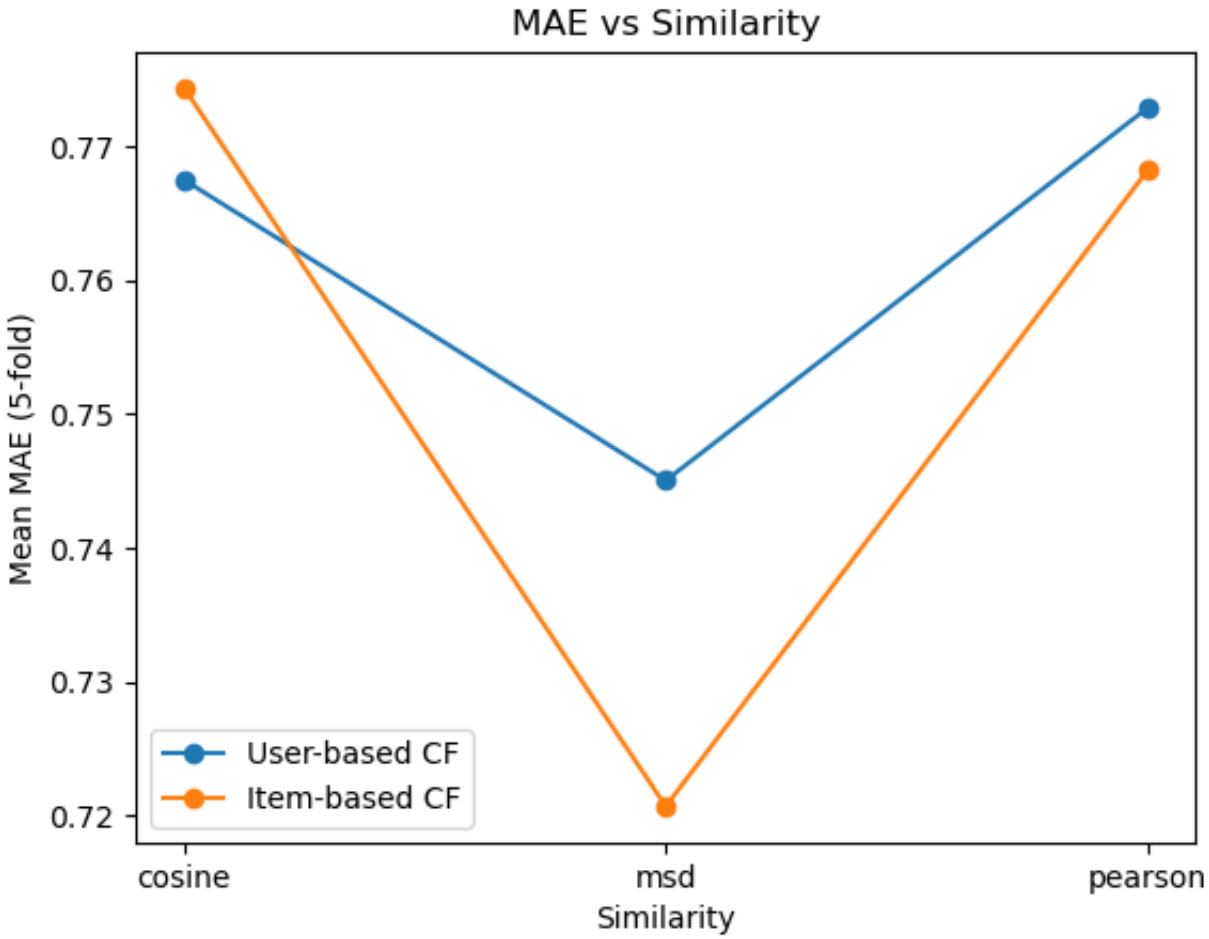
```
for model_name in ["User-based CF", "Item-based CF"]:
    sub = df_e[df_e["Model"] == model_name]
    plt.plot(sub["Similarity"], sub["RMSE_mean"], marker="o", label=model_name)
plt.xlabel("Similarity")
plt.ylabel("Mean RMSE (5-fold)")
plt.title("RMSE vs Similarity")
plt.legend()
plt.show()

# Plotting MAE
plt.figure()
for model_name in ["User-based CF", "Item-based CF"]:
    sub = df_e[df_e["Model"] == model_name]
    plt.plot(sub["Similarity"], sub["MAE_mean"], marker="o", label=model_name)
plt.xlabel("Similarity")
plt.ylabel("Mean MAE (5-fold)")
plt.title("MAE vs Similarity")
plt.legend()
plt.show()
```

```
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
```

```
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
Done computing similarity matrix.
Computing the pearson similarity matrix...
```

```
Done computing similarity matrix.
          Model Similarity  RMSE_mean  MAE_mean
0  User-based CF     cosine   0.993263  0.767468
1  Item-based CF     cosine   0.995754  0.774328
2  User-based CF        msd   0.969089  0.745014
3  Item-based CF        msd   0.934230  0.720670
4  User-based CF    pearson   0.999016  0.772851
5  Item-based CF    pearson   0.990230  0.768185
```





From your 5-fold results and plots, the similarity choice clearly affects both CF methods. For User-based CF, MSD similarity gives the lowest RMSE and MAE, while cosine is slightly worse and Pearson is the worst. For Item-based CF, the pattern is the same: MSD is best (lowest errors), cosine next, Pearson worst. **So yes, the impact is consistent across user-based and item-based CF**—all three similarities rank in the same order for both models (MSD best → cosine middle → Pearson worst), and both curves show a similar "dip at MSD, higher at cosine/Pearson" shape.

In [28]:
```
# f
```

In [30]:
```
# Trying a range of neighbor counts
K_values = [5, 10, 20, 30, 40, 50, 60, 80, 100]

def eval_knn_for_k(k, user_based_flag, sim_name="msd"):
    sim_options = {"name": sim_name, "user_based": user_based_flag}
    algo = KNNBasic(k=k, sim_options=sim_options)
    cv_res = cross_validate(algo, data, measures=["RMSE", "MAE"], cv=5, verbose=False)
    return np.mean(cv_res["test_rmse"]), np.mean(cv_res["test_mae"])

rows = []
for k in K_values:
    rmse_u, mae_u = eval_knn_for_k(k, True,  "msd")   # User-based
    rmse_i, mae_i = eval_knn_for_k(k, False, "msd")   # Item-based
    rows.append(["User-based CF", k, rmse_u, mae_u])
    rows.append(["Item-based CF", k, rmse_i, mae_i])

df_f = pd.DataFrame(rows, columns=["Model", "K_neighbors", "RMSE_mean", "MAE_mean"])
print(df_f)

# Ploting RMSE vs K
plt.figure()
for model_name in ["User-based CF", "Item-based CF"]:
    sub = df_f[df_f["Model"] == model_name]
    plt.plot(sub["K_neighbors"], sub["RMSE_mean"], marker="o", label=model_name)

plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Mean RMSE (5-fold)")
plt.title("RMSE vs Number of Neighbors")
plt.legend()
plt.show()

# Plot MAE vs K
plt.figure()
for model_name in ["User-based CF", "Item-based CF"]:
    sub = df_f[df_f["Model"] == model_name]
    plt.plot(sub["K_neighbors"], sub["MAE_mean"], marker="o", label=model_name)

plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Mean MAE (5-fold)")
```

```
plt.title("MAE vs Number of Neighbors")
plt.legend()
plt.show()
```

```
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
```

```
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
```

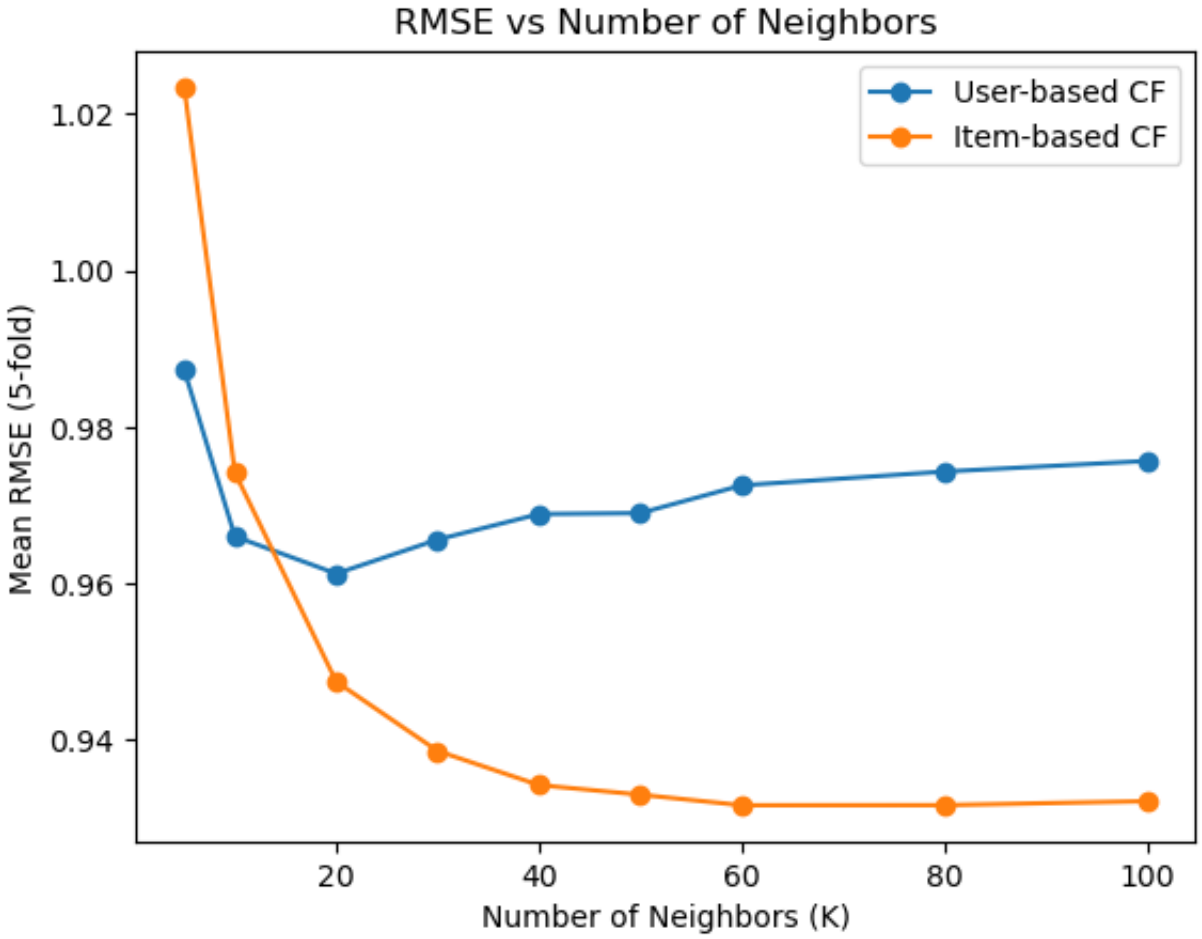
```
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
```

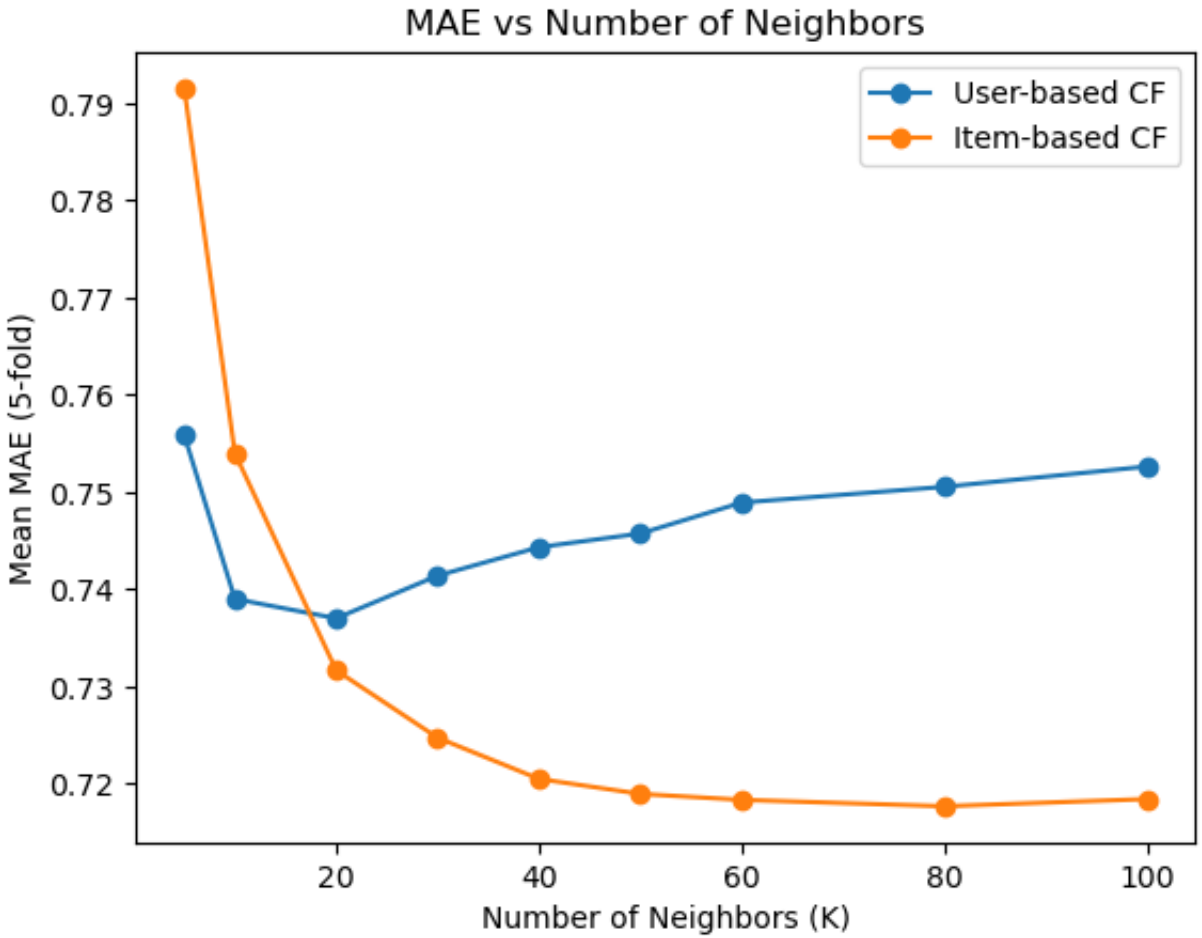
```
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
```

```
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
Computing the msd similarity matrix...
Done computing similarity matrix.
```

	Model	K_neighbors	RMSE_mean	MAE_mean
0	User-based CF	5	0.987319	0.755873
1	Item-based CF	5	1.023420	0.791551
2	User-based CF	10	0.966005	0.738994
3	Item-based CF	10	0.974169	0.753986
4	User-based CF	20	0.961252	0.736999
5	Item-based CF	20	0.947445	0.731646
6	User-based CF	30	0.965604	0.741387
7	Item-based CF	30	0.938588	0.724709
8	User-based CF	40	0.968867	0.744317
9	Item-based CF	40	0.934225	0.720468
10	User-based CF	50	0.968990	0.745724

11	Item-based CF	50	0.932998	0.718927
12	User-based CF	60	0.972537	0.748908
13	Item-based CF	60	0.931647	0.718314
14	User-based CF	80	0.974296	0.750532
15	Item-based CF	80	0.931660	0.717657
16	User-based CF	100	0.975653	0.752624
17	Item-based CF	100	0.932166	0.718397





As K increases, **User-based CF improves up to a moderate K, then slowly worsens**, showing there's an optimal middle range and too many neighbors add noise. **Item-based CF improves as K grows and then levels off**, meaning more neighbors help until performance stabilizes. Overall, user-based prefers a moderate K, while item-based benefits from larger K before plateauing.

In [33]:
```
# g
```

From the previous K-neighbors results table, we selected the K that gives the minimum RMSE for each method. For **User-based CF**, the lowest RMSE occurs at **K = 20**, and for **Item-based CF**, the lowest RMSE occurs at **K = 60**.

Therefore, the best K values are **not the same** for user-based and item-based collaborative filtering.